

Title: From Legacy Software Curriculum to AI-Era Program Design

Subtitle: A Bridge Model for Moving from Conventional Software Programs to AI-Era Workforce Preparation

1. Executive Summary

This white paper proposes a practical transition model for modernizing software curriculum in response to emerging AI-era workforce realities.

The Challenge

Wisconsin technical college software-related programs were largely designed in a pre-AI context. That does not mean those programs have lost their value, but it does mean they were not originally built for a workforce in which software development now occurs across multiple coexisting realities: conventional development, AI-assisted development, AI-injected application development, and AI-embedded systems operating on organizational data and workflow.

The current challenge, therefore, is not simply to add AI topics to an existing software curriculum. The deeper challenge is to redesign software education so that students can remain employable in conventional environments while also developing the capability to work responsibly and effectively in AI-shaped development contexts. If AI modernization is framed too narrowly as tool use, prompting, or a single machine learning course, the resulting curriculum will likely undershoot the actual shift taking place in software practice.

The Bridge Approach

This white paper argues for a practical transition model rather than a purely greenfield replacement approach. Under current statewide governance realities, Southwest Tech is using the approved IT Software Developer program as a bridge structure for AI-injected redesign. This bridge approach allows the college to begin integrating AI-era capabilities now while also generating concrete design evidence that can inform a future, more coherent proposed Associates AI Software Engineering (AAISE) successor program.

This transition logic is important because the redesign problem is not only curricular. It is also operational. Current and future program models may overlap for at least a year. Students may need to move between pathways. Faculty need a realistic implementation path. Institutions need a model that is not only conceptually stronger, but also administratively workable and clear enough to support review, advising, and adoption.

Why the Current Work Matters

Southwest Tech's current bridge work suggests that meaningful progress can begin inside existing structures without waiting for full statewide redesign. Early course-level work is already showing practical patterns that matter:

- foundational content can be preserved while AI is introduced in bounded and governed ways
- students can be taught to distinguish different roles for AI in their workflow
- data, APIs, architectural thinking, and justification of AI use can enter earlier than traditional software curricula often allow

These patterns are not yet full proof of final curriculum effectiveness, but they do provide grounded direction for further development.

Three Core Ideas

This paper proposes that statewide software-program modernization should be guided by three ideas.

First, software curriculum must now account for multiple Workforce Models rather than assume one stable development environment.

Second, a bridge-first strategy offers a practical path for modernizing current programs without pretending the final AI-era program architecture is already settled.

Third, future redesign should be governed by graduate capability targets rather than by inherited “teach everything” coverage habits that overload curriculum without necessarily improving integrative readiness.

What This Adds

Under this model, the bridge pathway preserves strong software foundations while expanding the curriculum to include:

- governed AI-assisted development
- AI role selection and justification
- modern data modeling
- AI-injected application design
- collaborative delivery workflow
- stronger systems and architectural thinking

At the same time, the bridge can reveal where the current program structure still falls short and where a future proposed Associates AI Software Engineering (AAISE) successor program would benefit from cleaner sequencing, more coherent course grouping, and a more explicit AI-era identity.

Why It Matters Now

The opportunity before Wisconsin technical colleges is not merely to insert AI into legacy curriculum. It is to design a more relevant and durable software education model for a mixed workforce future.

Southwest Tech’s current bridge effort may offer a practical pilot logic for that work: preserve what still matters, modernize what no longer fits, and use structured transition rather than abrupt replacement to move toward a stronger statewide model.

Contents

1. Executive Summary
2. The Current Problem State
3. Workforce Models for the AI Era
4. What Has Changed in Software Practice
5. Why Traditional “Add AI” Approaches Are Not Enough
6. A Practical Transition Model: Bridge First, Successor Next
7. What Southwest Tech Is Already Learning
8. Proposed Graduate Capability Direction
9. Illustrative Developmental Progression Across the Program
10. Illustrative Bridge Course Direction
11. Why This Matters for Student Mobility and Institutional Transition
12. Why Southwest Tech’s Work Could Serve as a Useful Pilot Model
13. Design Framework and Guiding Logic
14. Near-Term Recommendations
15. Conclusion
16. Appendix A. Graduate Capability Model Summary
17. Appendix B. Workforce Model Definitions
18. Appendix C. Condensed Bridge Course Direction Matrix

Intended Audience

This paper is written primarily for:

- WTCS program leaders and reviewers
- local supervisors and administrators
- advisory stakeholders
- curriculum designers and faculty engaged in software-program redesign

2. The Current Problem State

Wisconsin technical college software-related programs were largely designed for a pre-AI development environment. That does not mean those programs no longer have value. It does mean they were not originally structured for a workforce in which AI now affects software work at multiple levels, including development workflow, application behavior, internal business systems, and architectural decision-making.

The current statewide conversation already reflects this recognition. The issue is no longer whether change is needed. The more important question is what kind of change is actually required.

The shallow version of the problem is that current software programs are missing AI topics. The deeper version is that the programs were designed around an earlier model of software practice, while students are now entering a field in which multiple modes of practice coexist at the same time.

This means the redesign challenge is not simply to add content. It is to prepare students for a mixed workforce reality while still working inside present governance, advisory, and implementation constraints.

In that sense, the problem is both educational and operational.

Educationally, the curriculum must change because the field has changed.

Operationally, the change cannot be treated as if Wisconsin technical colleges are starting from a blank slate. Existing programs remain in place. Current and future program models may overlap. Students may move through more than one curricular version during the transition. Faculty need an implementation path that is realistic rather than purely aspirational.

The actual design problem, therefore, is not merely how to add AI to a software program. It is how to create a responsible transition from legacy-approved software curriculum to a more relevant AI-era model without breaking employability, coherence, or institutional practicality.

3. Workforce Models for the AI Era

One reason software-program redesign can become confused is that the phrase “AI in curriculum” is too broad. It can mean many different things, and if those meanings are not separated, the conversation often collapses into narrow tool-use assumptions.

For this reason, it is useful to distinguish among four Workforce Models that now shape software work.

Native

This is conventional software development without official AI participation in either the product or the approved development workflow.

In some organizations, AI use may be culturally, managerially, or operationally discouraged even if unofficial shadow use still occurs. This remains a real workforce condition, which means students still need strong conventional software capability.

AI-Assisted

This is work in which AI supports the human workflow but is not part of the delivered system itself.

This includes familiar low-barrier use cases such as drafting, summarization, search, and research support. In software contexts, it also includes coding assistance, debugging support, documentation drafting, comparison of options, and other forms of development acceleration.

AI-Injected

This is work in which AI is intentionally introduced into the application or solution architecture as part of the system's designed behavior or supported workflow.

This is not merely attaching a chatbot or adding a superficial AI feature. It implies purposeful integration in which the role, boundaries, and value of AI are architecturally justified.

This category can also include emerging agentic patterns in which AI participates in multi-step operations, orchestrated tool use, or governed action flows inside a designed system context.

AI-Embedded

This is work in which AI becomes part of the internal operating logic of the system or business workflow itself, especially in relation to organizational data, analysis, prediction, transformation, decision support, or generative operations.

Here AI is not peripheral. It participates directly in how the system functions internally.

Why These Distinctions Matter

These models matter because they explain why a modern software program cannot be designed around only one future condition.

Graduates may enter:

- conventional software teams
- organizations using AI mainly as workflow assistance
- environments building AI-enabled applications
- environments using AI internally inside business processes and data systems

If curriculum redesign ignores that mixed reality, it will likely prepare students either too narrowly or too abstractly.

4. What Has Changed in Software Practice

AI is not changing software work in only one way.

It is changing how software is written, how applications are conceived, how teams work, how data is used, how architecture is evaluated, and how quickly new abstractions move older manual work under the hood.

This matters because many existing curriculum assumptions were formed in an environment where:

- programming work was more manually centered
- tool chains were less AI-mediated
- application logic was less influenced by probabilistic or model-based components
- data interaction was often taught as a narrower database topic rather than as part of a larger architectural system

That environment no longer describes the field well enough on its own.

At the same time, the field is not moving uniformly. Many businesses are still adopting AI slowly. Some continue to operate in mostly conventional ways. Some are using AI primarily as a productivity layer. Others are beginning to redesign applications and internal systems around AI capabilities more directly.

This uneven adoption is precisely why software education must now prepare students for coexistence across multiple Workforce Models rather than assuming a single clean destination.

Another major change is pedagogical.

Traditional IT pedagogy has often assumed that strong preparation means exposing students to many technologies and manual procedures in isolated form. In practice, this can become a “teach everything” model that overloads the curriculum while failing to build enough integrative capability. Many tasks that once had to be taught as explicit manual procedure now sit under higher-level abstractions, frameworks, commands, libraries, and AI-assisted workflows.

This does not remove the need for understanding. It changes where the educational emphasis should sit.

Students still need conceptual grounding. They do not necessarily need every historical manual procedure taught as if the field had not changed.

That shift has major implications for curriculum design.

5. Why Traditional “Add AI” Approaches Are Not Enough

Once the problem is framed correctly, it becomes clear why shallow modernization strategies are unlikely to be sufficient.

The first common mistake is to treat AI as a later add-on topic. Under that model, the existing software curriculum remains largely untouched and AI is inserted through one or two advanced courses, a tool module, or a machine learning elective. This approach may create awareness, but it does not necessarily change the student’s relationship to workflow, architecture, data, systems reasoning, or professional judgment.

The second mistake is to reduce AI preparation to tool familiarity. Students may be shown how to use chatbots, coding assistants, or prompt techniques, but they may not be taught how to govern those tools, justify their use, validate their outputs, or distinguish between AI as helper, AI as architectural component, and AI as internal system logic.

The third mistake is inherited coverage inflation. A curriculum can become so committed to broad isolated coverage that it continues teaching every layer of manual detail even after the field has shifted toward higher-level abstractions, framework support, and integrated development environments. This often creates overloaded programs that feel comprehensive without actually becoming more coherent.

A stronger approach is to redesign around capability value.

That means asking:

- what should graduates actually be able to do?
- what kinds of environments should they be able to work in?
- what conceptual understanding still matters even when implementation details are increasingly abstracted?
- what kinds of manual depth still need to be preserved, and where?

This is why simple AI insertion is not enough.

The challenge is not only to introduce new topics. It is to realign curriculum structure, sequence, emphasis, and integration so that the program reflects the field students are actually entering.

6. A Practical Transition Model: Bridge First, Successor Next

If the field has changed, and if shallow curriculum updates are insufficient, the next question becomes how to redesign responsibly under current conditions.

The most practical answer is not immediate full replacement. It is a transition model.

This paper proposes a two-track approach.

Track 1. AI-Injected Bridge Revision of the Current Program

The first track uses the current approved IT Software Developer program as a bridge structure for meaningful AI-injected modernization.

Its purpose is to:

- preserve the usable value of the existing approved program
- begin integrating AI-era capabilities now
- support students who still need a clear path through the current structure
- generate practical design evidence from real curriculum work

This track is not merely cosmetic renaming. It involves substantive reframing of courses, earlier introduction of AI-assisted and AI-injected concepts, and stronger integration of data, workflow, architecture, and justification of AI use.

Track 2. Future Proposed Associates AI Software Engineering (AAISE) Successor Program

The second track is the design of a more coherent future program rather than a simple patching of the legacy structure.

Its purpose is to create a degree architecture intentionally aligned with the mixed software reality described earlier in this paper.

This future proposed Associates AI Software Engineering (AAISE) model would benefit from:

- cleaner sequencing
- more coherent course grouping
- stronger alignment to capability targets
- less dependence on legacy course-shell assumptions

Why the Two Tracks Should Stay Connected

These tracks should not be treated as separate unrelated projects.

The bridge can inform the future program by revealing:

- what maps well into the current structure
- what becomes overloaded
- what should be preserved
- what likely needs a cleaner long-term design

The future successor model can also inform the bridge by clarifying which capabilities and patterns matter enough to begin now, even before the final program architecture exists.

Why This Transition Model Matters

This approach addresses both educational and operational reality.

Educationally, it avoids pretending that minor AI insertion is enough.

Operationally, it avoids pretending that the final statewide solution can be designed and implemented all at once without attention to overlap, transition, and governance.

Most importantly, it creates a way to move forward now while still making room for a more coherent future program later.

That is the central value of the bridge model: it allows modernization to begin under current constraints while building toward a stronger AI-era program architecture rather than remaining trapped inside legacy assumptions.

7. What Southwest Tech Is Already Learning

One of the strengths of Southwest Tech's current position is that this work is no longer purely conceptual.

The redesign effort is still early, and it should be described honestly as early-stage bridge work rather than as a fully validated model. At the same time, meaningful instructional patterns are already emerging through actual course redesign activity, especially in the first-semester software sequence.

Current work suggests that foundational course content does not need to be abandoned in order to begin AI-era modernization. Instead, the stronger pattern is to preserve foundational programming, web, and algorithmic learning while changing the context in which students encounter it.

This means students can begin in a recognizably rigorous software pathway while also being introduced to:

- bounded AI-assisted workflow
- differentiated roles for AI in their work
- early data and API awareness
- architectural framing and justification of AI use

That matters because it suggests the bridge model can start much earlier in the program than a traditional “AI later” approach would allow.

Southwest Tech’s emerging bridge work also suggests that AI integration becomes more educationally useful when it is governed rather than unrestricted. Students are not simply being shown AI tools. They are being introduced to questions such as:

- what role should AI play here?
- where is AI helpful and where is it not?
- what still requires human judgment?
- how should AI use be justified?

This is a stronger pattern than shallow tool exposure because it begins to build professional judgment alongside technical capability.

At the same time, the current work is also exposing structural realities that matter for the larger statewide conversation. It is showing where the current approved program can carry the bridge effectively, where the structure begins to overload, and where a future Associates AI Software Engineering (AAISE) successor program would likely benefit from cleaner sequencing and more explicit AI-era design logic.

That makes Southwest Tech’s current work useful not because it is finished, but because it is already producing grounded design evidence.

8. Proposed Graduate Capability Direction

If the redesign effort is to become more than course renaming, it must be guided by a clear understanding of what graduates should actually be able to do.

The emerging capability direction suggests that software graduates now need more than conventional programming skill plus light AI familiarity.

They need durable software foundations, but they also need the ability to work responsibly across multiple Workforce Models. That means they should leave with capability in areas such as:

- governed AI-assisted development
- intentional selection and justification of AI use
- AI-injected application design
- awareness of AI-embedded business systems
- modern data modeling and retrieval reasoning
- systems and architectural thinking
- performance and abstraction awareness
- collaborative delivery workflow
- applied integration and demonstrated capability

This capability model matters because it shifts the redesign question from:

“What technologies should we add?”

to:

“What kinds of professional capability should a graduate be able to demonstrate in the workforce now emerging?”

That is a more stable design basis than topic insertion alone.

It also helps explain why some inherited curriculum habits may need to change. If the goal is capability, not coverage, then every course does not need to preserve every historical manual procedure or isolated technology exposure in the same way.

Instead, the program can begin asking:

- what conceptual depth still matters?
- what kinds of systems reasoning now matter more than isolated syntax recall?
- what modern workflow capabilities are now essential to employability?
- what must students be able to justify, not just perform?

This shift from topic coverage to capability development is one of the most important design changes in the entire proposal.

9. Illustrative Developmental Progression Across the Program

One of the clearest strengths of the current bridge design work is that it suggests a coherent developmental arc across the program rather than a scattered set of AI insertions.

The current progression can be described in four broad stages.

Semester 1: Foundations and Controlled AI Entry

The student begins by learning to code, reason, and work with AI under control.

The first semester preserves foundational programming and problem-solving work while introducing students to bounded AI-assisted workflow and the idea that AI must be governed rather than treated as magic.

Identity statement:

I can code, think, and begin using AI under control.

Semester 2: Data, Tools, and Modern Development Depth

The second semester deepens software fluency while making the bridge's modern direction more visible. Data systems, modern tooling, Python depth, and conventional employability pathways become more explicit.

Identity statement:

I understand data, tools, and modern development pathways more deeply.

Semester 3: Architecture, Deployment, and Systems Thinking

The third semester moves students more clearly into AI-injected design territory. Cloud, containers, systems analysis, architectural framing, and systems-level tradeoffs become more central.

Identity statement:

I can reason about AI as part of architecture, deployment, and system design.

Semester 4: Delivery, Integration, and Demonstrated Capability

The fourth semester functions as a professional synthesis layer. Students move from exposure and isolated capability toward delivery, integration, justification, and demonstration.

Identity statement:

I can deliver, integrate, justify, and demonstrate applied capability.

These identity statements matter because they show that the bridge is not merely adding AI vocabulary. It is creating a developmental progression that is both pedagogically coherent and professionally legible.

10. Illustrative Bridge Course Direction

Although this paper is not intended to function as a full curriculum handbook, it is still useful to show what the bridge direction looks like in practical terms.

At a high level, the bridge course logic appears to follow a selective pattern:

- preserve and reframe the strongest foundational software courses
- modernize the data and workflow spine
- merge or remove redundant or weak-fit legacy slots
- repurpose open curricular space toward higher-value bridge capability

Under this approach, the current program does not need to be discarded wholesale.

Instead, it can be modernized through a combination of:

- early-course reframing
- explicit AI-assisted workflow development
- stronger systems and architectural thinking
- modern data-modeling emphasis
- collaborative delivery workflow
- more bounded and meaningful culminating experiences

The bridge work also suggests that not all changes should be treated equally. Some current courses appear to function well as keep-and-reframe anchors. Others appear more useful as merger, replacement, or redesign opportunities. This is a strength rather than a weakness. It reflects the reality that thoughtful curriculum redesign does not assume every inherited course shell deserves equal long-term weight.

For management and advisory purposes, the practical point is simple:

the bridge does not require rebuilding the full program from scratch.

It requires selective modernization governed by capability value and workforce relevance.

11. Why This Matters for Student Mobility and Institutional Transition

A stronger curriculum model is not enough on its own. The transition also has to work institutionally.

This matters because current and future program structures may overlap for at least one year. During that period:

- some students may be finishing the legacy pathway
- some may be entering a revised bridge pathway
- some may need to move between structures
- faculty may be teaching across more than one curricular state at once

If transition logic is ignored, even a strong redesign can become difficult to implement in practice.

This is why the bridge model has value beyond pedagogy alone.

It provides a way to modernize the current program without forcing an abrupt break between what exists now and what may come next. It also helps create a clearer basis for advising, pathway explanation, and eventual alignment between current and future program forms.

For students, this matters because transition friction can become a hidden cost.

For institutions, it matters because adoption becomes much harder when redesign logic does not account for overlap, pacing, and implementation reality.

In other words, transition design is not a side issue. It is part of the architecture.

12. Why Southwest Tech's Work Could Serve as a Useful Pilot Model

Southwest Tech's current position is especially valuable because it combines three qualities that do not always appear together.

First, the college is working inside current constraints rather than treating constraints as a reason to wait.

Second, the work is not purely conceptual. Real bridge redesign activity is already underway.

Third, the design logic emerging from that work appears portable even if other colleges implement it differently.

This is why Southwest Tech's effort may serve as a useful pilot model.

The point is not that every college should copy Southwest Tech course for course.

The more important point is that the transition logic itself may be useful:

- preserve what still carries value
- modernize what no longer fits
- distinguish between bridge and successor design
- use real curriculum work to inform future program architecture

This is exactly the kind of practical pilot logic that can help a statewide conversation move from general agreement to usable direction.

13. Design Framework and Guiding Logic

The curriculum direction proposed in this paper is not random topic expansion.

It is guided by a design logic that can be described in practical terms without requiring broad prior agreement on new terminology.

That logic includes several key ideas:

- define the real problem before proposing the solution
- make constraints explicit early
- distinguish between bridge design and successor-program design
- allow lower-level curriculum work to revise higher-level understanding when reality reveals hidden constraints or better structure
- optimize for coherence rather than topic accumulation

This matters because curriculum work often fails not from lack of ambition, but from weak governing structure.

If the frame is vague, the redesign becomes shallow.

If the frame is too rigid, the redesign becomes trapped inside its own early assumptions.

The strongest design posture is neither unrestricted experimentation nor rigid top-down planning. It is a governed process in which practical artifact work can refine the larger architecture without dissolving coherence.

That is part of why the current bridge effort is valuable. It is functioning not only as curriculum work, but also as a discovery process for what a stronger software-program architecture should become.

14. Near-Term Recommendations

Based on the work to date, several near-term recommendations appear justified.

1. Continue AI-injected bridge development within the current approved program

This allows meaningful modernization to proceed now rather than waiting for full successor-program approval.

2. Use the bridge as a learning structure, not merely as an interim label

Bridge work should continue to generate real instructional and design evidence that can inform later statewide and successor-program decisions.

3. Align redesign around graduate capabilities rather than inherited coverage habits

This will help prevent modernization from collapsing into topic accumulation or tool demonstration.

4. Preserve conventional employability while expanding AI-era relevance

Students still need to succeed in conventional software environments even as the curriculum expands toward AI-assisted, AI-injected, and AI-embedded work.

5. Encourage pilot-oriented statewide collaboration

If multiple colleges are now asking similar questions, coordinated pilot logic may be more useful than disconnected local experimentation.

6. Reserve room for a cleaner future Associates AI Software Engineering (AAISE) model

The bridge should not be mistaken for the final form. Its value lies partly in what it reveals about the architecture of the future program.

15. Conclusion

The question facing Wisconsin technical colleges is no longer whether software curriculum must change in response to AI.

That question has already been answered by the field itself.

The more important question is how to redesign responsibly.

This paper argues that responsible redesign requires more than AI topic insertion. It requires a transition architecture that recognizes multiple Workforce Models, preserves employability, modernizes capability development, and works within the institutional realities that colleges actually face.

A bridge-first model offers a practical path forward. It allows meaningful work to begin now inside current structures while also generating insight that can inform a stronger future Associates AI Software Engineering (AAISE) successor program.

Southwest Tech's current work suggests that this kind of transition is not only possible, but already underway in usable form.

That creates an opportunity not simply to react to AI, but to help shape a more relevant and durable model of software education for the mixed workforce future that students are now entering.

The immediate opportunity is to continue structured bridge work now while collaboratively shaping the future program model across the Wisconsin technical college system.

Appendices

The following appendices provide supporting reference material on graduate capability direction, Workforce Model definitions, and the condensed bridge course structure.

Appendix A. Graduate Capability Model Summary

This appendix summarizes the graduate capability direction proposed for the AI-injected bridge pathway and the future Associates AI Software Engineering (AAISE) successor model.

It is included because program redesign within a competency-oriented review environment requires more than course renaming. It requires a coherent logic for what graduates should be able to do.

The capability model below is not yet a final WIDS competency map. It is a governing capability framework intended to support later development of course competencies, learning objectives, and assessment strategies.

1. Software Foundations

Graduates should be able to:

- frame problems before jumping to solutions
 - reason through logic, flow, and program behavior
 - write, read, and revise code with purpose
 - debug and improve software methodically
 - communicate technical intent clearly
-

2. Governed AI-Assisted Development Practice

Graduates should be able to:

- use AI for coding, debugging, drafting, documentation, research, and analysis
 - refine prompts, context, and constraints to improve results
 - compare options rather than accept the first plausible answer
 - detect weak, shallow, or overconfident AI output
 - remain accountable for final work
-

3. AI Role Selection and Justification

Graduates should be able to:

- distinguish between AI as explainer, assistant, and collaborator
 - decide when one role is more appropriate than another
 - explain why AI was used in a given task or project
 - identify where AI should not be used
 - justify accepted outputs in relation to evidence, fit, and constraints
-

4. AI-Injected Application Design

Graduates should be able to:

- identify when AI is and is not an appropriate addition to a system
- define the intended role of AI within an application or workflow

- distinguish between a shallow AI feature and meaningful AI-enabled design
 - reason about boundaries, fallback behavior, and failure modes
 - understand that AI injection changes architecture, testing, and support needs
-

5. AI-Embedded Systems Awareness

Graduates should be able to:

- understand descriptive, predictive, and generative AI use in practical contexts
 - reason about internal AI-supported workflows rather than only public-facing features
 - identify governance, privacy, reliability, and oversight concerns
 - understand the difference between a demo feature and an operationally useful AI-enabled business process
-

6. Modern Data Modeling and Retrieval Reasoning

Graduates should be able to:

- reason about relational and non-relational data models
 - understand when different data approaches fit different problems
 - recognize what ORMs and abstractions hide and reveal
 - understand why vector and embedding-oriented patterns matter in some AI contexts
 - connect data modeling choices to retrieval quality and application behavior
-

7. Architectural and Systems Thinking

Graduates should be able to:

- define what a system is trying to achieve before selecting implementation details
 - identify components, boundaries, dependencies, and flows
 - distinguish between symptoms and underlying causes
 - evaluate tradeoffs rather than default to convenience
 - reason upward from implementation issues to architectural implications
-

8. Systems Performance and Abstraction Awareness

Graduates should be able to:

- reason about memory behavior and performance implications
 - recognize tradeoffs between convenience and control
 - understand abstraction boundaries and their consequences
 - connect higher-level design decisions to lower-level system behavior
-

9. Testing, Validation, and Uncertainty Handling

Graduates should be able to:

- test software behavior systematically
 - validate AI-assisted or AI-generated outputs against requirements and evidence
 - recognize when certainty is not yet justified
 - design fallback paths, review loops, and bounded failure handling
 - communicate limitations and uncertainty honestly
-

10. Collaborative Delivery and CI/CD Workflow

Graduates should be able to:

- use version control in a disciplined way
 - work with branches, pull requests, and review processes
 - understand collaborative workflow and issue coordination
 - recognize the role of automation and CI/CD in delivery quality
 - use AI assistance inside governed team workflow
-

11. Human Governance and Professional Judgment

Graduates should be able to:

- maintain accountability for decisions and delivered work
 - decide when to accept, revise, reject, or escalate AI-supported output
 - recognize bias, overreach, and false confidence in generated material
 - preserve ethical, legal, and professional boundaries
-

12. Applied Integration and Demonstrated Capability

Graduates should be able to:

- combine software foundations, AI-assisted workflow, and architectural reasoning in one coherent artifact or system slice
 - justify key design choices
 - demonstrate what was built and why
 - show how testing, validation, and governance were handled
 - connect technical decisions to practical outcomes
-

13. Adaptability and Continuous Learning

Graduates should be able to:

- acquire new tools and concepts without losing grounding
 - ask useful technical questions
 - compare alternatives and make informed choices
 - update approaches when evidence or conditions change
 - move between conventional, transitional, and more AI-intensive environments
-

Summary Interpretation

Taken together, these capability groups suggest that software-program redesign should be governed by what students must be able to do across multiple workforce conditions rather than by inherited assumptions about isolated topic coverage.

This capability model is therefore intended to function as a precursor to later:

- course competencies
- learning objectives
- assessment strategies
- WIDS-aligned curriculum mapping

Appendix B. Workforce Model Definitions

This appendix provides a concise reference for the four Workforce Models used in the white paper.

These distinctions matter because the phrase “AI in software education” is too broad on its own. Different workforce conditions require different kinds of preparation.

1. Native

Conventional software development without official AI participation in either the delivered product or the approved development workflow.

This remains a real workforce condition and still requires strong conventional software capability.

2. AI-Assisted

Software work in which AI supports the human workflow but is not part of the delivered system itself.

This includes uses such as:

- coding assistance
 - debugging support
 - documentation drafting
 - summarization
 - research and search support
 - comparison of options
-

3. AI-Injected

Software work in which AI is intentionally introduced into the application or solution architecture as part of the system’s designed behavior or supported workflow.

This is more than adding a superficial AI feature. It implies purposeful integration in which the role, boundaries, and value of AI are architecturally justified.

This can also include emerging agentic patterns in which AI participates in multi-step operations, orchestrated tool use, or governed action flows within a designed system context.

4. AI-Embedded

Software work in which AI becomes part of the internal operating logic of the system or business workflow itself, especially in relation to organizational data, analysis, prediction, transformation, decision support, or generative operations.

Here AI is not peripheral. It participates directly in how the system functions internally.

Why These Definitions Matter

These Workforce Models help clarify why a modern software program cannot be designed as if there were only one stable future condition.

Graduates may enter environments that range from fully conventional to strongly AI-enabled. A useful software curriculum should therefore prepare students for movement across these conditions rather than for only one narrow interpretation of AI.

Appendix C. Condensed Bridge Course Direction Matrix

This appendix provides a condensed semester-by-semester summary of the proposed AI-injected bridge direction for the current Southwest Tech IT Software Developer program.

It is included as a practical reference for management, advisory, and review discussion.

Semester 1

Current Course	Bridge Action	Bridge Direction	Optional / AI-Aligned Title
10-151-101 Introduction to Security	Review	Keep, reframe, or replace later if needed	TBD
10-152-117 Python Programming	Keep + Reframe + Expand	Foundational programming with staged AI-assisted integration already in progress	Foundations of Programming for AI-Augmented Engineers
10-152-118 HTML, CSS & JavaScript	Keep + Reframe	Web foundations with early AI-assisted workflow integration	Web Foundations for AI-Augmented Engineers
10-152-119 Introduction to Algorithms	Keep + Reframe	Algorithmic thinking, logic discipline, and AI-aware problem solving	Algorithmic Thinking for AI-Augmented Engineers

Semester 2

Current Course	Bridge Action	Bridge Direction	Optional / AI-Aligned Title
10-152-120 SQL Programming	Keep + Reframe	Keep practical SQL foundations in the bridge sequence	SQL Programming
10-152-121 Advanced Python Programming	Keep + Reframe + Expand	Jupyter notebooks, Python libraries, ML workflow, data handling, visualization, Django exposure	Advanced Python Systems for AI-Augmented Engineers
10-152-122 C# Programming	Keep + Reframe	Preserve conventional software-development employability with C#/.NET and folded-in ASP.NET exposure	C# and .NET Application Development
10-152-123 SQL Programming Advanced	Replace / Redesign	Replace with broader modern data-system reasoning; bridge may keep this after SQL even if future AAISE reorders it earlier	Modern Data Modeling for AI-Enabled Systems

Semester 3

Current Course	Bridge Action	Bridge Direction	Optional / AI-Aligned Title
10-152-124 Introduction to Docker and Containers	Keep + Reframe	Containerization and deployment literacy for modern application and AI-enabled systems	Containerized Application Deployment
10-152-125 ASP.NET Programming	Merge / Replace	Merge core ASP.NET exposure into C# and repurpose this slot	Likely Collaborative Delivery, GitHub, and CI/CD
10-152-126 Cloud for Developers	Keep + Reframe + Expand	Cloud context, service integration, and infrastructure awareness for AI-enabled systems	Cloud Platforms for AI-Enabled Systems

Current Course	Bridge Action	Bridge Direction	Optional / AI-Aligned Title
10-152-127 AI-Driven Systems Analysis	Keep + Expand	Primary carrier for systems analysis, AI role definition, and full architectural framing of AI-enabled systems	Applied AI Systems Analysis
10-152-128 C++ Programming	Keep + Radical Reframe	Reposition around memory, performance tradeoffs, abstraction boundaries, and systems-level reasoning; optionally expose LibTorch/TensorFlow lightly	Systems Performance and AI Runtime Foundations
10-152-132 Mobile Development	Review	Retain only if aligned to a clearer stack and workforce purpose	Possible Flutter/Dart Intelligent App Development

Semester 4

Current Course	Bridge Action	Bridge Direction	Optional / AI-Aligned Title
10-152-129 Agile Practices	Keep + Reframe + Expand	Expand to include GitHub workflow, pull requests, review, and CI/CD	Agile Delivery for AI-Enabled Teams
10-152-130 Software Career Experience	Replace	Replace long, loose end-of-program experience with a bounded applied integration course	Applied AI Integration Studio
10-152-131 Machine Learning and AI	Keep + Reframe	Preserve bounded ML/deep learning exposure with PyTorch/TensorFlow orientation and support for internal AI-enabled system development	Machine Learning Foundations and Applications
10-152-133 Enterprise Data Systems & API Architecture	Remove / Replace	Current form appears over-scoped or redundant for the bridge	TBD

Quick Read Summary

Likely keep-and-reframe anchors

- 10-152-117 Python Programming
- 10-152-118 HTML, CSS & JavaScript
- 10-152-119 Introduction to Algorithms
- 10-152-120 SQL Programming
- 10-152-121 Advanced Python Programming
- 10-152-122 C# Programming
- 10-152-124 Introduction to Docker and Containers
- 10-152-126 Cloud for Developers
- 10-152-127 AI-Driven Systems Analysis
- 10-152-129 Agile Practices
- 10-152-131 Machine Learning and AI

Likely merge, remove, or replace candidates

- 10-152-123 SQL Programming Advanced
- 10-152-125 ASP.NET Programming
- 10-152-130 Software Career Experience
- 10-152-133 Enterprise Data Systems & API Architecture

Review-heavy candidates

- 10-151-101 Introduction to Security

- 10-152-128 C++ Programming
- 10-152-132 Mobile Development